

Healthcare Management System REST API

Enterprise Software Engineering Project Report & Architecture Guide

Project Author: Nandennagari Haasini	Target Environment: Java 21 / Spring Boot 4.1.x
Repository: github.com/nandennagarihaasini/Healthcare	Security: Spring Security (HTTP Basic Auth)
Documentation: Springdoc OpenAPI / Swagger UI 2.8.6	Database: Relational (MySQL / Embedded H2)

1. Project Overview

The **Healthcare Management System REST API** is an enterprise-grade backend service engineered to modernize and digitize operations in hospitals, clinics, and outpatient healthcare networks. It provides a robust, centralized, and decoupled platform for managing the entire patient care lifecycle across six critical business domains: **Patients, Doctors, Appointments, Prescriptions, Medical Records, and Billing**.

Problem Statement: Traditional healthcare administration relies on disparate systems—paper files, isolated billing spreadsheets, and disconnected scheduling registers. This creates critical operational bottlenecks: patient history fragmentation, consultation scheduling conflicts, unfulfilled prescriptions, and revenue leakages. **Solution:** This project establishes a unified, transaction-safe RESTful API adhering to industry-standard HTTP methods, strict relational integrity via Spring Data JPA, automated data validation, and real-time Swagger documentation.

2. Functional Modules

The architecture is organized around six autonomous yet interconnected functional modules:

Module	Primary Responsibilities	Key Endpoints
Patient Management	Patient onboarding, demographics tracking, contact details, and admission timeline.	/api/patients GET, POST, PUT, DELETE
Doctor Management	Directory of medical staff, departmental specialization, phone, and professional email.	/api/doctors GET, POST, PUT, DELETE
Appointment Scheduling	Relational nexus linking patient and doctor; enforces future date and entity existence validation.	/api/appointments GET, POST, PUT, DELETE
Prescription Issuance	1:1 clinical prescription generated per appointment; records medication name, dosage, and intake advice.	/api/prescriptions GET, POST, PUT, DELETE
Medical Records (EHR)	Cumulative electronic health records, clinical diagnoses, and prescribed treatments linked to patients.	/api/records GET, POST, PUT, DELETE
Billing & Invoicing	1:1 financial invoices per consultation tracking invoice amount, payment status, and timestamp.	/api/billing GET, POST, PUT, DELETE

3. Technology Stack

Category	Technology Selected	Architectural Rationale
Language	Java 21 (LTS)	Strong typing, high concurrency, virtual threads, and enterprise ecosystem.
Framework	Spring Boot 4.1.x / 3.x	Auto-configuration, production-ready metrics, embedded Tomcat server.
Persistence / ORM	Spring Data JPA / Hibernate	Eliminates repetitive SQL; automated schema generation and dirty checking.
Databases	MySQL 8.0 & H2 In-Memory	MySQL for enterprise persistent storage; H2 for zero-dependency local execution.
Build Automation	Apache Maven 3.9+	Standardized dependency management and reproducible multi-platform builds.
Code Optimization	Project Lombok	Reduces boilerplate code via @Data, @Builder, and @RequiredArgsConstructor.
Data Validation	Jakarta Bean Validation	Declarative constraints (@NotBlank, @FutureOrPresent) preventing corrupt data.
Security	Spring Security	HTTP Basic Authentication, CSRF protection, and endpoint role access control.
API Documentation	Springdoc OpenAPI 2.8.6	Dynamic OpenAPI 3 schema and live, interactive browser-based Swagger UI.
API Testing	Postman Tool	Collection-based test automation, request payloads, and status code verification.

4. External Dependencies

Key Maven dependencies specified in pom.xml:

- **spring-boot-starter-webmvc**: Core REST framework containing Spring MVC, Jackson JSON serializer, and embedded Apache Tomcat.
- **spring-boot-starter-data-jpa**: Integrates Hibernate ORM, HikariCP connection pooling, and Spring Data repository proxies.
- **spring-boot-starter-validation**: Hibernate Validator implementation of Jakarta Validation 3.0 specification.
- **spring-boot-starter-security**: Secures REST endpoints using Basic Auth and provides SecurityFilterChain bean customization.
- **springdoc-openapi-starter-webmvc-ui (v2.8.6)**: Automatically generates /v3/api-docs and mounts interactive Swagger UI.
- **mysql-connector-j & h2**: High-performance JDBC drivers for production MySQL and embedded in-memory H2 databases.
- **lombok**: Compile-time annotation processor for getters, setters, constructors, and builder patterns.

5. Architecture

The system follows a strict **Layered Architecture Pattern** with separation of concerns:



HTTP Request (JSON Payload + Basic Auth) ▼

CONTROLLER LAYER: @RestController, @RequestMapping, @Valid ■ ■ PatientController | DoctorController | AppointmentController | Billing etc. ■

Validated Domain Entities / DTOs ▼

```
SERVICE LAYER: @Service, Business Validation, Transaction Boundaries ■ ■ PatientService | DoctorService |
AppointmentService | PrescriptionService ■
```

Spring Data JPA Method Invocations ▼

```
REPOSITORY LAYER: JpaRepository, Derived Queries, ORM Mapping ■ ■ PatientRepository | DoctorRepository | AppointmentRepository etc. ■
```

SQL Execution via HikariCP Pool ▼

DATABASE LAYER: Relational Tables (patients, doctors, appointments, etc.) ■

Entity Relationships: Patient (1:M) Appointment | Doctor (1:M) Appointment | Appointment (1:1) Prescription | Patient (1:M) MedicalRecord | Appointment (1:1) Billing. Centralized error interception is governed by GlobalExceptionHandler.

6. Folder Structure

Clean package hierarchy organized by architectural layer:

```
D:\Healthcare project ■■■■ .git/ # Git repository metadata ■■■■
Healthcare_Management_System_Documentation.pdf # Generated technical reference PDF ■■■■
PROJECT_DOCUMENTATION.md # Technical project documentation ■■■■ UPLOAD_TO_GITHUB.bat # 1-Click
interactive GitHub sync tool ■■■■ postman/ # Postman workspace & collections ■■■■
collections/Healthcare REST API/ # YAML requests for all 6 modules ■■■■ healthcare/ # Spring Boot
Application Root ■■■■ pom.xml # Maven build descriptor & dependencies ■■■■ mvnw / mvnw.cmd #
Cross-platform Maven wrappers ■■■■ README.md & INTERVIEW_GUIDE.md # Project overview & interview Q&A; ■
■■■■ src/main/java/com/example/healthcare/ ■■■■ HealthcareApplication.java # Spring Boot main entry point
■■■■ config/ ■■■■ SecurityConfig.java # Basic Auth & Swagger permitAll rules ■■■■
DataInitializer.java # Automated database seeder (Dr. Haasini ID: 3) ■■■■ controller/ # 6 REST API
Controllers (@RestController) ■■■■ service/ # 6 Business Service classes (@Service) ■■■■ repository/ # 6
JPA Repositories (JpaRepository) ■■■■ entity/ # 6 Relational Entities (@Entity) ■■■■ exception/ #
GlobalExceptionHandler & ResourceNotFoundException
```

7. Development Methodology

The project was engineered using an **Agile, Feature-Driven Iterative Lifecycle**. Rather than attempting monolithic construction, features were developed in vertical testable slices:

- **Phase 1 — Inception & Scaffolding:** Spring Initializr setup, Maven dependencies, MySQL/H2 configuration.
- **Phase 2 — Core Master Entities:** Constructed Patient and Doctor modules with Repository-Service-Controller chains.
- **Phase 3 — Relational Nexus:** Implemented Appointment module connecting Patients and Doctors with foreign keys.
- **Phase 4 — Clinical & Billing Operations:** Constructed 1:1 Prescription, 1:M Medical Records, and 1:1 Billing modules.
- **Phase 5 — Defensive Programming:** Implemented Jakarta Validation and global exception handling with clean JSON payloads.
- **Phase 6 — Security & Documentation:** Configured Spring Security Basic Auth, Swagger UI, and automated data seeding.
- **Phase 7 — Verification & Release:** Executed comprehensive Postman testing, PDF generation, and GitHub version control sync.

8. Testing & Quality Assurance

Quality assurance was conducted across positive, negative, and edge-case execution matrices using Postman and Swagger UI:

Test Category	Endpoint / Scenario Tested	Input Condition	Expected Result
Positive CRUD	POST /api/patients	Valid patient JSON	201 Created + ID
Entity Lookup	GET /api/doctors/3	Query ID = 3 (Dr. Haasini)	200 OK + Details
Validation Edge	POST /api/appointments	Past appointmentDate	400 Bad Request
Constraint Check	POST /api/patients	Missing contact field	400 + Field Error
Not Found Exception	GET /api/doctors/999	Non-existent ID	404 Not Found
Security Gate	GET /api/billing	No credentials	401 Unauthorized
Unique 1:1 Rule	POST /api/billing	Duplicate appointment ID	Database Rejection

9. Building & Running

Follow these instructions to compile, run, and test the project locally:

1. CLONE THE REPOSITORY: `git clone https://github.com/nandennagarihaasini/Healthcare.git` `cd Healthcare/healthcare` 2. RUN THE APPLICATION (Using embedded Maven Wrapper): Windows: `.\mvnw.cmd clean spring-boot:run` Linux/Mac: `./mvnw clean spring-boot:run` 3. ACCESS ENDPOINTS & DOCUMENTATION: - Base REST API URL: `http://localhost:8080/api/` - Interactive Swagger UI: `http://localhost:8080/swagger-ui.html` - OpenAPI v3 JSON Schema: `http://localhost:8080/v3/api-docs` - H2 In-Memory Database Web: `http://localhost:8080/h2-console` 4. AUTHENTICATION CREDENTIALS (Spring Security Basic Auth): - Username: `admin` - Password: `admin123`

10. Conclusion

The **Healthcare Management System REST API** represents a complete, production-ready backend solution for medical center administration. Developed using **Java 21**, **Spring Boot**, **Spring Data JPA**, and **Spring Security**, the project demonstrates best practices in enterprise software engineering: decoupled layered design, strict referential integrity, declarative bean validation, centralized error handling, and automated interactive API documentation.

Key Takeaways: The modular architecture ensures seamless scalability and maintainability. Future expansions—such as JWT token authentication, Role-Based Access Control (RBAC), cloud attachment storage for lab reports, and automated payment gateways—can be integrated with zero breaking changes to existing client integrations.

Repository: <https://github.com/nandennagarihaasini/Healthcare>

Status: Production Ready / Tested ■